

Vector Storage and Embeddings

1. What Embeddings Are: The Mathematical Foundation

An embedding is a fixed-length list of numbers, called a vector, that represents a piece of text, an image, or other data in a high-dimensional mathematical space. The key property of embeddings is that proximity in this space reflects semantic similarity: concepts that are related in meaning are represented by vectors that are close together, while unrelated concepts are far apart.

Embedding models are neural networks trained on vast corpora of text. During training, the network learns to map words, sentences, and passages to positions in the vector space such that items appearing in similar contexts cluster together. The resulting space encodes relationships that go far beyond word frequency: 'cyberattack,' 'security breach,' and 'network intrusion' will all occupy a region of the space associated with security incidents, even though they share no words.

For security practitioners, this semantic awareness is the critical enabler of intelligent retrieval. A traditional system searching for 'ransomware playbook' might miss a document titled 'Crypto-locker Response Procedure.' An embedding-based system finds it, because both descriptions occupy nearby regions in embedding space. This is not magic; it is the direct consequence of how the embedding model was trained.

Memory anchor: Think of embedding space as a vast city map where similar ideas live in the same neighbourhood. 'Ransomware,' 'crypto-locker,' and 'file encryption attack' all live on the same street, even if they look different on paper. Distance on the map is semantic distance, not alphabetical distance.

2. How Embedding Models Capture Meaning

Concept	Explanation	Security Example
Contextual similarity	Words appearing in similar contexts get similar embeddings through training.	'Malware' and 'virus' appear in similar technical contexts, so their embeddings are close.
Phrase-level semantics	Embedding models handle multi-word phrases, not just individual words.	'Advanced persistent threat' and 'nation-state espionage campaign' map to nearby vectors despite sharing no words.
Cross-lingual transfer	Multilingual models produce embeddings where the same concept in different languages sits nearby.	A threat report in French and its English equivalent retrieve the same relevant incident records.
Negation limitation	Standard embedding models struggle with negation: 'no breach detected' may be close to 'breach detected.'	Filtering on structured metadata or post-processing is needed when negation is semantically critical.

Understanding these properties helps you select the right embedding model and anticipate failure modes. For security text, models fine-tuned on cybersecurity corpora (such as SecBERT or security-domain variants of BGE) typically outperform general-purpose models on retrieval benchmarks involving threat terminology, CVE descriptions, and attack-pattern language.

3. Embedding Dimensions and Quality Trade-offs

Every embedding model produces vectors of a fixed number of dimensions. OpenAI's text-embedding-3-large produces 3072-dimensional vectors. Cohere Embed v3 produces 1024 dimensions. Sentence-BERT models commonly use 384 or 768 dimensions. These numbers matter for three reasons: storage cost, computation cost, and retrieval quality.

- **Higher dimensions:** Can encode more nuanced semantic distinctions, potentially improving retrieval precision. Also require more disk space and more compute per similarity calculation.
- **Lower dimensions:** Faster to store and search, especially at scale. Quality reduction is often negligible when the model was well trained at that dimension count.
- **Matryoshka embeddings:** Some modern models (including OpenAI text-embedding-3 series) support truncating embeddings to smaller sizes while retaining most quality, giving you a performance-versus-quality dial without switching models.

In practice, choose embedding models based on quality benchmarks on tasks similar to yours, such as the MTEB (Massive Text Embedding Benchmark), rather than selecting by dimension count alone. A 768-dimensional model that was trained on security text will often outperform a 3072-dimensional general-purpose model on security retrieval tasks.

4. Creating and Storing Embeddings: The Ingestion Pipeline

Building a RAG knowledge base requires running every source document through an ingestion pipeline before the system can answer any questions. The pipeline has several stages that must be carefully designed.

- **Document loading:** Source documents are ingested from wherever they live: SharePoint, S3 buckets, ticketing systems, threat intelligence platforms. Connectors parse formats including PDF, DOCX, HTML, JSON, and plain text.
- **Chunking:** Each document is split into passages of appropriate size, typically 256 to 1024 tokens, with optional overlap (10 to 20 percent) to avoid cutting concepts mid-sentence.
- **Metadata enrichment:** Each chunk is tagged with structured metadata: document title, source system, creation date, last-modified date, document type, author, security classification level, and any domain-specific tags such as affected CVEs or threat actor names.
- **Embedding generation:** Each chunk is passed through the embedding model via API call or local inference, producing a vector. The same model must be used for ingestion and for query-time embedding; mixing models breaks similarity comparisons.
- **Storage:** Vectors and their associated metadata and original text are written to the vector database. The original chunk text is needed to pass to the LLM as context; only the vector is used for similarity search.

Critical rule: The embedding model used at ingestion time **MUST** be the same model used at query time. Changing embedding models requires re-embedding the entire knowledge base. This is an operational decision, not a minor configuration detail.

5. Vector Storage: Why Traditional Databases Fall Short

Storing a million 1024-dimensional vectors requires roughly 4 GB of float32 memory. That is manageable. The harder problem is search: finding the nearest neighbours to a query vector among a million candidates. A naive approach that computes cosine similarity between the query and every stored vector would require a million dot-product operations per query. At scale, that becomes prohibitively slow.

Traditional relational databases are optimised for exact lookups by indexed columns, not approximate nearest-neighbour search in high-dimensional space. Even PostgreSQL with the pgvector extension, while functional for small to medium deployments, cannot match the throughput of purpose-built vector databases at millions of vectors and hundreds of queries per second.

Storage Option	Best For	Limitations
In-memory (flat)	Development, datasets under ~1M vectors, maximum accuracy.	RAM-limited; no persistence by default; expensive at large scale.
Disk-based with HNSW index	Production workloads requiring fast approximate search with high recall.	Index build time is significant; recall is approximate, not exact.
Disk-based with IVF index	Very large datasets where partitioning reduces search space.	Requires clustering step at index build; slightly lower recall than HNSW at same speed.
Hybrid memory/disk	Large-scale production; hot vectors in RAM, cold on disk.	Complexity of cache management; most managed vector databases handle this automatically.

6. Vector Indexing: HNSW and IVF Explained

The two dominant index structures in production vector databases are HNSW (Hierarchical Navigable Small World) and IVF (Inverted File Index). Understanding their mechanics helps you reason about performance trade-offs without memorising benchmark tables.

- **HNSW:** Builds a layered graph where each node connects to its nearest neighbours. Search navigates the graph from a coarse top layer down to a fine bottom layer, skipping large regions of the space at each step. Delivers high recall at fast query speeds. Used by Weaviate, Qdrant, pgvector, and others. Main drawback: high memory usage because the graph structure must be held in RAM.
- **IVF (Inverted File Index):** Clusters vectors into groups (Voronoi cells) at index build time. A query first identifies which clusters are most relevant, then searches only those clusters. Much lower memory overhead than HNSW. Main drawback: if the relevant vector falls in a cluster that was not selected, it is missed entirely, reducing recall. Used by FAISS and as a component in many vector databases.

- **Product Quantisation (PQ):** A compression technique that approximates vectors using codebooks, dramatically reducing storage and computation at the cost of some accuracy. Often combined with IVF as IVF-PQ for large-scale deployments where memory is constrained.

For most security RAG deployments, HNSW is the default choice because recall is critical: missing a relevant incident report or threat intelligence entry has real operational consequences. IVF-PQ is appropriate when the knowledge base scales into the hundreds of millions of vectors and memory costs become prohibitive.

7. Similarity Metrics: Cosine, Euclidean, and Dot Product

Metric	What It Measures	When to Use
Cosine similarity	The angle between two vectors regardless of their magnitude. Ranges from -1 (opposite) to 1 (identical direction).	Most common for text embeddings. Normalisation makes magnitude irrelevant; only semantic direction matters.
Dot product	The product of magnitudes times cosine similarity. Magnitude matters.	Used when embedding model is trained with dot-product objective (e.g., some bi-encoder models). Faster than explicit cosine when vectors are not normalised.
Euclidean distance (L2)	Straight-line distance between two points in the vector space.	Less common for text; useful when embedding magnitude carries semantic information, e.g., image embeddings.

The correct metric depends on how the embedding model was trained. Using the wrong metric produces incorrect similarity scores and degraded retrieval quality. Always check the model card for the recommended similarity metric and configure your vector database accordingly.

8. Popular Vector Databases for Security Deployments

Database	Deployment Model	Key Strengths	Typical Use Case
Pinecone	Managed cloud (SaaS)	Zero infrastructure management; fast setup; automatic scaling.	Teams wanting fastest time-to-value; not suitable for air-gapped environments.
Weaviate	Self-hosted or managed cloud	Rich GraphQL API; built-in hybrid search; strong metadata filtering.	Enterprise deployments needing on-premises option with full feature set.
Milvus	Self-hosted	Designed for billion-scale workloads; Kubernetes-native.	Large organisations with very large document collections and dedicated infrastructure.
Chroma	Self-hosted	Extremely simple API; minimal	Developers building and

	(embedded)	setup; good for development and prototyping.	testing RAG pipelines before production sizing.
Qdrant	Self-hosted or managed cloud	High performance; strong Rust-based core; good sparse vector support for hybrid search.	Production deployments prioritising query speed and hybrid search capability.
PostgreSQL + pgvector	Self-hosted (existing DB)	Leverages existing PostgreSQL infrastructure; ACID compliance; familiar tooling.	Organisations with existing PostgreSQL and small to medium vector workloads.

Real-World Incident: LastPass Breach and Knowledge Base Security (2022-2023): The LastPass breach of 2022 exposed encrypted password vaults along with unencrypted metadata. Subsequent analysis showed that even metadata, such as URLs and account names, provided attackers with significant intelligence for targeting high-value accounts. This is directly analogous to the risk of storing sensitive security documents in a RAG knowledge base without appropriate access controls and encryption. If a vector database containing your organisation's incident reports, vulnerability assessments, and network architecture documents were compromised, the attacker would gain a structured, searchable map of your security posture. Vector databases must be treated as sensitive data stores with encryption at rest, strict access controls, and audit logging, not as read-only reference systems.

9. Metadata Design for Security Knowledge Bases

Metadata is what separates a naive document dump from a queryable, filterable, auditable knowledge base. Every chunk stored in a vector database should carry structured metadata that enables both operational filtering and governance requirements.

- **Temporal fields:** `document_created_date`, `document_last_modified`, `ingested_at`. Essential for freshness filtering. Use ISO 8601 format for portability.
- **Source and provenance:** `source_system`, `document_id`, `document_title`, `author`, `url_or_path`. Required for citation and audit trail.
- **Classification and sensitivity:** `security_classification` (Public, Internal, Confidential, Restricted), `data_sensitivity_tier`. Enables access-control filtering so users only retrieve documents appropriate to their clearance level.
- **Domain-specific tags:** `threat_actor`, `affected_cve`, `affected_platform`, `incident_id`, `regulatory_framework`. Enables precise pre-filtering before semantic search runs, dramatically improving result relevance.
- **Document type:** `playbook`, `threat_intelligence`, `policy`, `incident_report`, `cve_description`. Allows type-specific queries such as 'show only playbooks, not general information.'

Investing in metadata schema design before ingestion is far less expensive than retrospectively re-tagging a knowledge base of 500,000 chunks. Define your metadata fields as part of system architecture, not as an afterthought.

10. Operational Best Practices for Vector Storage in Security

- **Backup and disaster recovery:** A vector database containing your institutional knowledge is critical infrastructure. Implement regular snapshots, test restoration procedures, and include vector database recovery in your disaster recovery plan.
- **Access control:** Apply least-privilege access to both the vector database API and the underlying storage. Service accounts used by the RAG application should have read-only access to the vector database; only the ingestion pipeline needs write access.
- **Encryption at rest and in transit:** All vectors, metadata, and stored chunk text should be encrypted at rest. All traffic between the RAG application, embedding service, and vector database should be TLS-encrypted.
- **Audit logging:** Log all queries to the vector database including the query vector, returned chunk IDs, and the requesting identity. This log supports both security incident investigation and retrieval quality auditing.
- **Index maintenance:** HNSW indexes degrade in recall quality as vectors are deleted and re-inserted over time. Schedule periodic index rebuilds to maintain retrieval performance.
- **Capacity planning:** Model vector count growth based on document ingestion rates. A corpus that doubles in size annually requires storage and index capacity to scale proportionally. Plan ahead to avoid emergency scaling events.

Key Terms Glossary

Term	Definition
Embedding	A fixed-length numerical vector representing text or other data in a high-dimensional space where proximity encodes semantic similarity.
Embedding model	A neural network trained to convert text (or other data) into embedding vectors; must be the same model at ingestion and query time.
Vector database	A database specialised for storing embedding vectors and performing fast approximate nearest-neighbour similarity search.
Dimensions	The number of numerical values in an embedding vector; common values are 384, 768, 1024, and 3072.
HNSW (Hierarchical Navigable Small World)	A graph-based vector index structure that enables fast approximate nearest-neighbour search with high recall, at the cost of elevated memory usage.
IVF (Inverted File Index)	A clustering-based vector index that partitions vectors into cells and searches only the most relevant clusters, using less memory than HNSW but with lower recall.
Cosine similarity	A similarity metric measuring the angle between two vectors; the most common metric for text embeddings.
Approximate nearest-neighbour (ANN) search	Finding the K most similar vectors without exhaustively comparing against every stored vector; the fundamental operation of vector databases.

Metadata filtering	Applying structured field constraints (e.g., date range, document type) to narrow the candidate set before similarity search runs.
Product Quantisation (PQ)	A compression technique that approximates embeddings using codebooks, reducing storage and computation at the cost of some accuracy.
Chunk	A passage of text produced by splitting a source document; the unit stored and retrieved in a RAG knowledge base.
Ingestion pipeline	The automated workflow that loads documents, chunks them, generates embeddings, enriches metadata, and writes to the vector database.

Quick Revision Checklist

- Embeddings are fixed-length vectors where semantic similarity corresponds to vector proximity.
- The same embedding model must be used at both ingestion and query time; mixing models breaks similarity comparisons.
- Higher embedding dimensions can capture more nuance but cost more in storage and compute; choose by quality benchmarks, not dimension count.
- Traditional databases cannot efficiently perform approximate nearest-neighbour search at scale; dedicated vector databases are required for production RAG.
- HNSW: high recall, high memory. IVF: lower recall, lower memory. Choose based on scale and recall requirements.
- Cosine similarity is the most common metric for text embeddings; always match the metric to the embedding model's training objective.
- Metadata design is critical: temporal fields enable freshness filtering; classification fields enable access control; domain tags enable precision pre-filtering.
- Vector databases are critical infrastructure: apply encryption at rest and in transit, least-privilege access, audit logging, and backup procedures.
- HNSW indexes require periodic rebuilds to maintain recall quality as vectors are deleted and re-added.
- For air-gapped deployments, use self-hosted vector databases (Weaviate, Milvus, Qdrant) and self-hosted embedding models.

10 Exam-Based MCQs: Vector Storage and Embeddings

Test yourself after reading. These questions are written in real exam style, scenario-heavy with plausible distractors. Read every explanation, including for questions you answered correctly.

Q1. *Scenario:* Maya is the AI infrastructure lead at a financial services firm. The current embedding model used in the RAG threat intelligence system was trained two years ago and lacks coverage of recent AI-specific attack terminology. The team wants to upgrade to a newer model. Maya needs to brief the CISO on the operational impact before the change is approved. The security team plans to update the

organisation's embedding model to a newer, higher-quality version. What is the MOST critical operational implication of this change?

- A) The LLM must also be retrained to accommodate the new embedding format.
- B) The entire knowledge base must be re-embedded using the new model before similarity search will work correctly.
- C) Query latency will increase because the new model produces larger vectors.
- D) Metadata filters will need to be reconfigured to match the new embedding dimensions.

Answer: B **Explanation:** B is correct. Embeddings from different models occupy incomparable vector spaces. If ingestion used Model A and queries use Model B, similarity scores will be meaningless and retrieval will break. The only fix is to re-embed every chunk using the new model. A is wrong: the LLM is independent of the embedding model; it only sees retrieved text, not vectors. C is partially true (larger vectors do add latency) but is not the critical implication; broken similarity is. D is wrong: metadata filters operate on structured fields unrelated to embedding dimensions.

Q2. Scenario: During a post-incident review, the threat intelligence team discovers that their RAG system failed to surface three relevant incident records during the initial triage of a suspected supply chain compromise. The records existed in the knowledge base but were not returned in search results. A security analyst queries the RAG system with 'supply chain compromise via third-party software update' and receives results that match the exact phrase but miss relevant incidents described as 'SolarWinds-style attack' or 'trojanised update package.' What is the MOST likely root cause?

- A) The vector database's HNSW index has degraded and needs to be rebuilt.
- B) The embedding model is not capturing semantic similarity between these equivalent descriptions.
- C) The metadata filters are too restrictive, excluding relevant incident records.
- D) The context window is too small to include all relevant retrieved chunks.

Answer: B **Explanation:** B is correct. If semantically equivalent descriptions ('supply chain compromise via third-party software update', 'SolarWinds-style attack', 'trojanised update package') are not returning each other in similarity search, the embedding model is not placing them near each other in vector space. This typically means the model lacks fine-grained cybersecurity domain knowledge or was not trained on this terminology. A is wrong: HNSW degradation causes reduced recall uniformly, not selective misses on specific concepts. C is wrong: the question does not mention metadata filters; the issue is semantic retrieval. D is wrong: context window limits affect how many retrieved chunks are passed to the LLM, not what is retrieved in the first place.

Q3. Scenario: The SOC lead at an e-commerce company wants to reduce triage time during incidents by enabling analysts to query the AI assistant with specific constraints on incident severity, date ranges, and affected business systems. The SOC team needs a RAG knowledge base that can answer questions like 'show me only critical severity incidents from the last 30 days affecting our payment processing systems.' Which capability is MOST essential for this use case?

- A) A high-dimensional embedding model with at least 3072 dimensions.

- B) Structured metadata fields on each chunk combined with pre-filtering before semantic search.
- C) A re-ranking model to improve relevance of returned results.
- D) Conversational multi-turn support in the RAG interface.

Answer: B **Explanation:** B is correct. The query combines three constraints: severity level (critical), date range (last 30 days), and affected system (payment processing). These are structured attributes that semantic search alone cannot reliably enforce. Metadata fields for severity, ingestion date, and affected system, combined with pre-filtering before the vector similarity search runs, are the direct solution. A is wrong: embedding dimensions affect semantic nuance but not the ability to filter by structured attributes. C is wrong: re-ranking improves precision of semantically retrieved results but does not enforce structured constraints. D is wrong: multi-turn support enables follow-up questions but does not address structured filtering.

Q4. Which similarity metric is MOST appropriate for comparing text embeddings produced by a standard sentence embedding model?

- A) Euclidean distance.
- B) Manhattan distance.
- C) Cosine similarity.
- D) Jaccard similarity.

Answer: C **Explanation:** C is correct. Cosine similarity measures the angle between vectors, making it insensitive to vector magnitude and therefore appropriate for text embeddings where the direction (semantic content) matters more than the magnitude. A (Euclidean distance) is less common for text embeddings because it is sensitive to magnitude; it is more often used for image embeddings. B (Manhattan distance) is rarely used for high-dimensional text embeddings. D (Jaccard similarity) measures set overlap and is used for sparse representations, not dense neural embeddings.

Q5. An organisation's RAG knowledge base has grown to 50 million vectors. The team is experiencing query latency exceeding three seconds. Which index configuration change would MOST likely reduce latency while maintaining acceptable recall?

- A) Switch from HNSW to a flat index to avoid graph traversal overhead.
- B) Reduce the embedding dimension by switching to a smaller embedding model.
- C) Switch to an IVF index with product quantisation to reduce search space and vector size.
- D) Increase the number of vectors retrieved per query to amortise the latency cost.

Answer: C **Explanation:** C is correct. At 50 million vectors, IVF-PQ is designed precisely for this scale: IVF partitions the search space so only relevant clusters are searched, and product quantisation compresses vectors to reduce memory and computation per similarity calculation. The combination significantly reduces latency at the cost of a small, acceptable recall reduction. A is wrong: a flat index is slower at large scale, not faster; it compares the query against every stored vector. B might reduce per-vector computation but also degrades semantic quality; it is a last resort. D is wrong: retrieving more vectors per query increases latency, not decreases it.

Q6. Which statement BEST describes why vector databases are necessary for production RAG systems rather than standard relational databases?

- A) Relational databases cannot store numerical data required by embeddings.
- B) Vector databases provide ACID transaction guarantees that relational databases lack.
- C) Vector databases are optimised for approximate nearest-neighbour search in high-dimensional space, which relational databases cannot perform efficiently at scale.
- D) Relational databases do not support the JSON metadata formats used alongside embeddings.

Answer: C **Explanation:** C is correct. The core operation of RAG retrieval is finding the K most similar embeddings to a query vector among millions of candidates. This approximate nearest-neighbour search in high-dimensional space is the specialisation of vector databases. Standard relational databases have no efficient index structure for this operation. A is wrong: relational databases can store arrays of numbers. B is wrong: vector databases often sacrifice ACID guarantees for performance; relational databases typically have stronger ACID support. D is wrong: most relational databases (including PostgreSQL) support JSON storage natively.

Q7. What is the PRIMARY risk of failing to encrypt a vector database that contains embedded security documentation?

- A) Query performance degrades because decryption adds latency to every operation.
- B) Embedding model quality decreases because vectors are exposed to potential corruption.
- C) An attacker who gains access to the database can extract the stored chunk text and metadata, revealing sensitive security information.
- D) Compliance frameworks do not recognise vector databases as data stores requiring protection.

Answer: C **Explanation:** C is correct. Vector databases store not only embedding vectors but also the original chunk text and metadata. An attacker with database access can read incident reports, playbooks, vulnerability assessments, and network architecture documentation. Encryption at rest protects this data if the storage layer is compromised. A is wrong: encryption overhead is a performance consideration, not a risk; modern hardware encryption has minimal impact. B is wrong: encryption does not affect embedding model quality. D is wrong: data protection regulations apply to sensitive data regardless of storage format; security documentation is sensitive data.

Q8. A RAG system was deployed six months ago with high retrieval accuracy. Recently, analysts report that results quality has degraded, particularly for topics added to the knowledge base in the last two months. The embedding model and chunk size have not changed. What is the MOST likely cause?

- A) The LLM's context window has been reduced by a provider update.
- B) The HNSW index has degraded in recall quality due to accumulated insertions and deletions.
- C) The newer documents use longer sentences that exceed the embedding model's token limit.
- D) Cosine similarity scores have drifted due to vector space saturation.

Answer: B **Explanation:** B is correct. HNSW indexes are known to experience recall degradation over time as vectors are inserted and deleted without a full rebuild. The graph connections become suboptimal for the current vector distribution, causing the search to miss relevant items. This manifests as declining

quality particularly for recently added content. A is wrong: the LLM context window affects how much retrieved text can be processed, not what is retrieved. C is wrong: chunks from long sentences are truncated or handled by the chunking step; this would have been an issue from the start. D is wrong: 'vector space saturation' is not a recognised phenomenon in embedding retrieval systems.

Q9. Which chunking strategy is MOST appropriate for a security knowledge base containing multi-step incident response playbooks?

- A) Fixed-size chunks of 128 tokens to maximise retrieval precision.
- B) Whole-document chunks to preserve complete playbook context.
- C) Semantic chunks that preserve complete procedural steps, with overlapping windows to avoid cutting mid-procedure.
- D) Random chunks of variable size to ensure diverse retrieval coverage.

Answer: C **Explanation:** C is correct. Incident response playbooks contain structured procedures where context and sequence matter. Chunking must preserve complete steps to avoid returning a step without its preceding or following steps, which would produce incomplete and potentially dangerous guidance. Overlapping windows (a portion of text shared between adjacent chunks) further reduce the risk of splitting key procedures. A is wrong: 128-token chunks are very small and will frequently split procedures mid-step, losing critical context. B is wrong: whole-document chunks are too large; they dilute retrieval relevance and may exceed the LLM context window. D is wrong: random chunking has no basis in document structure and will produce incoherent retrieved passages.

Q10. An organisation's RAG system for a classified threat intelligence programme cannot use cloud-based services. Which combination BEST satisfies both capability and deployment requirements?

- A) OpenAI text-embedding-3-large for embeddings, Pinecone for vector storage, GPT-4o for generation.
- B) A self-hosted open-source embedding model such as BGE-M3, Milvus or Qdrant for vector storage, and a self-hosted Llama 3 or Mistral model for generation.
- C) Cohere Embed v3 for embeddings, Weaviate Cloud for vector storage, Claude 3 Sonnet via API for generation.
- D) Any combination is acceptable provided the data is encrypted in transit.

Answer: B **Explanation:** B is correct. Air-gapped or classified deployments prohibit sending sensitive data to external APIs. Self-hosted open-source embedding models (BGE-M3, E5-large, Sentence-BERT variants), self-hosted vector databases (Milvus, Qdrant, Weaviate self-hosted), and self-hosted LLMs (Llama 3, Mistral) are all available as on-premises deployments with no external network dependency. A is wrong: OpenAI embeddings, Pinecone, and GPT-4o all require internet API calls; data would leave the classified environment. C is wrong: Cohere, Weaviate Cloud, and Claude API all require external network connectivity. D is wrong: encryption in transit protects data in motion but does not prevent it from being transmitted to and processed by external services.